Lab 07 — Antwoorden met toelichting

Elk antwoord volgt hetzelfde schema: het antwoord, het mechanisme, de nuance of valkuil, en een voorbeeld dat je aan de terminal kunt nagaan.

Vraag 1 — `localhost:5432` in de API

Antwoord. In de container `api` wijst `localhost` naar **de container `api`**: daar luistert niemand op 5432, vandaar `Connection refused`. In de container `db` is `localhost:5432` wel degelijk PostgreSQL — maar de API zit daar niet. Correctie: `SPRING_DATASOURCE_URL=jdbc:postgresql://db:5432/app`, de containernaam, geresolved door de DNS van het user-defined netwerk.

Waarom. Elke container heeft zijn eigen network namespace, dus zijn eigen `lo`. `127.0.0.1` steekt nooit een namespacegrens over.

Nuance. `pg_isready` in `db` bewijst dat de database luistert, niet dat ze bereikbaar is vanuit `api`. De relevante test doe je **vanuit** `api` (of vanuit een wegwerpcontainer op hetzelfde netwerk).

Voorbeeld.

```
podman run --rm --network stack alpine nc -zv db 5432          # db (10.89.0.2:5432) open
podman run --rm --network stack alpine nc -zv localhost 5432   # Connection refused
```

Vraag 2 — Leeg `IPAddress`

Antwoord. De container zit op het **standaard rootless-netwerk**, beheerd door `pasta`, dat het adres van de host naar de container kopieert in plaats van er een toe te kennen: Podman heeft dus geen "container-IP" om te tonen. Een adres `10.88.0.x` krijg je door de container te starten met `--network podman` (de bridge), of beter, op een netwerk dat je zelf aanmaakt.

Waarom. Een gewone gebruiker kan geen bridge en geen NAT aanmaken; `pasta` doet alles in user space door de stack van de host na te bootsen. Rootless bridge-netwerken leven in een privé-namespace van Podman.

Nuance. `HostConfig.NetworkMode` is `pasta` in het eerste geval, `bridge` in het tweede. Docker als root toont altijd een IP, want zijn standaardnetwerk is al een bridge.

Voorbeeld.

```
podman run -d --name s alpine sleep 1d && podman inspect s --format
'{{.NetworkSettings.IPAddress}}|{{.HostConfig.NetworkMode}}'   # |pasta
podman run -d --name p --network podman alpine sleep 1d && podman inspect p --format
'{{.NetworkSettings.Networks.podman.IPAddress}}'  # 10.88.0.2
podman rm -f -t 0 s p
```

Vraag 3 — `ping` op IP, `ping` op naam

Antwoord. Op `podman`: de `ping 10.88.0.2` slaagt (zelfde bridge), de `ping a` faalt met `bad address 'a'`. Op `stack`: allebei slagen ze. Het verschil zit in het veld `dns_enabled` van `podman network inspect`: `false` op `podman`, `true` op een netwerk dat je zelf aanmaakt — dat start dan `aardvark-dns`.

Waarom. De bridge zorgt voor de IP-connectiviteit; naamresolutie is een aparte dienst, die Podman alleen op user-defined netwerken inschakelt (net als Docker).

Nuance. Zelfs op `stack` kan `ping` op naam falen in een image zonder de permissie `NET_RAW`; `wget`, `nc` of `getent hosts` zijn betrouwbaardere tests dan een `ping`.

Voorbeeld.

```
podman network inspect podman --format '{{.DNSEnabled}}'   # false
podman network inspect stack  --format '{{.DNSEnabled}}'   # true
podman run --rm --network stack alpine getent hosts api    # 10.89.0.3  api.dns.podman …
```

Vraag 4 — "`EXPOSE` dient nergens toe"

Antwoord. Hij heeft ongelijk in de grond van de zaak, gelijk in zijn waarneming. `EXPOSE` **publiceert geen** poort: het is metadata van de image die zegt "deze dienst luistert op 8080". Ze wordt gelezen door `-P` (dat alle exposed poorten op willekeurige hostpoorten publiceert), getoond door `podman ps` (`8080/tcp` zonder pijl) en `podman image inspect` (`ExposedPorts`), en dient als documentatie voor wie de image uitrolt.

Waarom. Publiceren is een beslissing bij de **uitrol** (welke poort van welke machine), niet bij het bouwen van de image. De image kan die niet nemen.

Nuance. Tussen containers van hetzelfde netwerk is `EXPOSE` noch `-p` nodig: alle poorten van de container zijn bereikbaar. `EXPOSE` blijft een goede gewoonte: tools (Compose, Kubernetes, scanners) gebruiken het.

Voorbeeld.

```
podman image inspect api:1.0 --format '{{.Config.ExposedPorts}}'   # map[8080/tcp:{}]
podman run -d --name r --network stack -P api:1.0 && podman port r  # 8080/tcp -> 0.0.0.0:39843
podman rm -f -t 0 r
```

Vraag 5 — Poort 80 geweigerd

Antwoord. Bij klassieke Docker opent de **root-daemon** poort 80: geen enkele beperking. In rootless is het jouw gebruiker, en de kernel houdt de poorten < 1024 voor root. Drie uitwegen: (1) publiceren op een hoge poort, `-p 8080:80`, en de URL met `:8080` aanvaarden; (2) de systeemgrens verlagen, `sudo sysctl net.ipv4.ip_unprivileged_port_start=80` (blijvend via `/etc/sysctl.d/`), wat ze voor alle gebruikers openzet; (3) er een reverse proxy of load balancer voor zetten die wél als root of als systeemdienst draait. In het bedrijf: (3) — een applicatiecontainer luistert nooit rechtstreeks op 80/443.

Waarom. De beperking is een regel van de Linux-kernel, ouder dan containers: een geprivilegieerde poort garandeerde vroeger dat de dienst door de beheerder gestart was.

Nuance. De container is wel aangemaakt maar niet gestart: `podman ps -a` toont hem als `Created`. Je moet hem verwijderen voor je met dezelfde naam opnieuw start.

Voorbeeld.

```
podman ps -a --filter name=web --format '{{.Status}}'   # Created
podman rm web && podman run -d --name web -p 8080:80 nginx:alpine
```

Vraag 6 — `502` nadat de API opnieuw aangemaakt is

Antwoord. `502 Bad Gateway`. nginx heeft `api` **bij het opstarten** geresolved naar `10.89.0.3` en houdt dat adres bij. De nieuwe `api` kreeg een ander IP (`andere` nam het volgende), het oude bestaat niet meer: `connect() failed (113: Host is unreachable)`. De wegwerpcontainer resolvet `api` op het moment van zijn verzoek en vindt het juiste adres. Herstel: `podman restart web` (nginx resolvet opnieuw bij het opstarten), of een nginx-configuratie met `resolver 10.89.0.1 valid=10s;` en `set $upstream http://api:8080; proxy_pass $upstream;`, die bij elk verzoek een resolutie afdwingt.

Waarom. Standaard resolvet nginx de namen van statische `proxy_pass`-regels bij het inlezen van de configuratie, één keer, om niet per verbinding een DNS-verzoek te betalen.

Nuance. Zolang `api` alleen **gestopt** is, is de code `504 Gateway Time-out` (het IP antwoordt niet meer); het is het **opnieuw aanmaken met een ander IP** dat `502` geeft. Spring Boot naar PostgreSQL heeft dat gebrek niet: elke nieuwe verbinding van de pool resolvet opnieuw.

Voorbeeld.

```
podman logs web 2>&1 | grep -i error | tail -1     # upstream: "http://10.89.0.3:8080/…"
podman inspect api --format '{{.NetworkSettings.Networks.stack.IPAddress}}'   # 10.89.0.12: een ander adres
podman restart -t 1 web && curl -s -o /dev/null -w '%{http_code}\n'
localhost:18080/api/actuator/health   # 200
```

Vraag 7 — Het clientadres is de container

Antwoord. In rootless wordt de gepubliceerde poort bediend door `rootlessport`, een doorgeefluik in user space dat **de verbinding op de host aanvaardt en een nieuwe opent** naar de container, vanuit de network namespace van Podman: nginx ziet dus als client het adres van de container zelf. Bij Docker als root wordt de poort doorgegeven via NAT-regels van de kernel, die het echte bronadres bewaren. Gevolg: elke logica op basis van het client-IP (logging, rate limiting, whitelist) is blind achter een rootless gepubliceerde poort.

Waarom. Een applicatief doorgeefluik (TCP-proxy) beëindigt de verbinding, terwijl NAT alleen pakketheaders herschrijft.

Nuance. Op het standaardnetwerk (`pasta`) blijft het bronadres beter bewaard. En in productie lost de header `X-Forwarded-For`, toegevoegd door de load balancer of de reverse proxy vooraan, het probleem op.

Voorbeeld.

```
podman logs web 2>&1 | grep GET | tail -1 | cut -d' ' -f1   # 10.89.0.10
podman inspect web --format '{{.NetworkSettings.Networks.stack.IPAddress}}'   # 10.89.0.10
```

Vraag 8 — Twee netwerken

Antwoord. Ja. Typisch geval: een netwerk `admin` voor monitoring- of back-uptools, waarop je `api` (of `db`) aansluit zonder `web`. Commando's: `podman network create admin`, `podman network connect --alias backend admin api`, `podman network disconnect admin api` om het ongedaan te maken. De inspect toont dan twee sleutels, `stack` en `admin`, elk met een eigen `IPAddress` (`10.89.0.12` en `10.89.1.3`) en eigen aliassen.

Waarom. Elk netwerk is een aparte bridge; een container aansluiten voegt een interface (`eth1`) toe in zijn namespace. De DNS van elk netwerk kent alleen zijn eigen leden.

Nuance. Een container van `admin` ziet `web` en `db` **niet**: de segmentatie is echt. Het is eerder een isolatiemiddel dan een gemak.

Voorbeeld.

```
podman network connect --alias backend admin api
podman inspect api --format '{{range $k,$v := .NetworkSettings.Networks}}{{$k}}=
{{$v.IPAddress}} {{end}}'   # admin=10.89.1.3 stack=10.89.0.12
podman run --rm --network admin alpine wget -qO- http://backend:8080/actuator/health
```

Vraag 9 — `--network host`

Antwoord. (1) **Geen `-p` meer**: de container luistert rechtstreeks op de poorten van de host, dus twee API's met `--network host` op 8080 botsen. (2) **Geen interne DNS en geen isolatie meer**: `api` resolvet `db` niet meer op naam, en de container ziet alle interfaces en alle poorten van de host. Gerechtvaardigd voor een netwerktool (`tcpdump`, scanner, monitoringagent) of voor extreme netwerkprestaties. In rootless faalt een applicatie die poort 80 wil, zoals elk proces van jouw gebruiker: `bind() to 0.0.0.0:80 failed (13: Permission denied)`.

Waarom. `--network host` maakt geen network namespace aan: het proces deelt die van de host, met dezelfde beperkingen en dezelfde conflicten.

Nuance. Wat de container in rootless `--network host` ziet, is de namespace van **jouw gebruiker** op de host, niet die van een daemon: in de praktijk identiek aan een proces dat zonder container gestart wordt.

Voorbeeld.

```
podman run --rm --network host alpine ip -4 -o addr show | grep -v ' lo '   # je WSL-interfaces
podman run --rm --network host nginx:alpine sh -c 'nginx; sleep 1'          # bind() …
Permission denied
```

Vraag 10 — Netwerk of pod

Antwoord. In het netwerk `stack` schrijft nginx `proxy_pass http://api:8080/` (naam geresolved door de DNS), en `-p 18080:80` staat op `web`. In de pod `stackpod` schrijft nginx `proxy_pass http://localhost:8080/` (zelfde namespace), en `-p 18090:80` declareer je **bij het aanmaken van de pod**; `podman ps --pod` toont een kolom `POD ID` en een container `…-infra` die de publicatie draagt. Dat tweede model is dat van **Kubernetes**: een Pod is daar een groep containers die netwerk en `localhost` delen, en de manifesten van deel 2 van deze reeks bouwen erop.

Waarom. De pod deelt één network namespace onder zijn leden; het netwerk verbindt aparte namespaces.

Nuance. In een pod kunnen twee containers niet op dezelfde poort luisteren: één `8080` per pod. En `/etc/hosts` van de pod vermeldt al zijn containers op `127.0.0.1`.

Voorbeeld.

```
podman pod create --name stackpod -p 18090:80
podman run -d --pod stackpod --name api-pod api:1.0
podman exec api-pod cat /etc/hosts | tail -2    # 127.0.0.1 api-pod …
```

Vraag 11 — DBeaver op `db`

Antwoord. `-p 127.0.0.1:5432:5432`: de poort staat alleen open op de loopback-interface van WSL. De browser of DBeaver onder Windows bereikt hem toch, omdat WSL 2 **elke poort waarop de distributie luistert doorstuurt naar `localhost` aan de Windows-kant**, ook de poorten die aan `127.0.0.1` gebonden zijn. Andere machines op het netwerk zien niets. In productie publiceer je de database nooit: ze is alleen bereikbaar voor de diensten van haar netwerk, en mensen gaan via een bastion of een speciale beheertool.

Waarom. Een poort publiceren is een deur openen op de machine; `127.0.0.1:` houdt ze voor lokale processen. Het WSL-doorgeefluik is een gemak van Windows, geen regel van Podman.

Nuance. In de *mirrored* netwerkmodus van WSL delen Windows en WSL hun interfaces, en zou een `-p 0.0.0.0:5432:5432` de database aan het lokale netwerk blootstellen. Hou `127.0.0.1:` als reflex.

Voorbeeld.

```
podman run -d --name db --network stack -p 127.0.0.1:5432:5432 -e POSTGRES_PASSWORD=secret
postgres:16-alpine
podman port db   # 5432/tcp -> 127.0.0.1:5432
```

Vraag 12 — `network rm` geweigerd

Antwoord. `-f` verwijdert eerst **alle containers en pods** die op het netwerk aangesloten zijn, en daarna het netwerk. In een opruimreeks is dat een slecht idee omdat je de lijst van wat gaat verdwijnen niet leest — en omdat een container tot dat netwerk kan behoren zonder dat je eraan denkt (een beheertool, een testdatabase). Juiste volgorde: `podman network inspect stack --format '{{range .Containers}}{{.Name}} {{end}}'` om te kijken, `podman rm -f -t 0 web api db` bij naam, en dan `podman network rm stack`.

Waarom. Een netwerk met leden is een resource in gebruik; Podman beschermt standaard, net als bij `volume rm`.

Nuance. Een **gestopte** container telt nog altijd als lid: het verwijderen faalt ook als er niets draait.

Voorbeeld.

```
podman network inspect stack --format '{{range .Containers}}{{.Name}} {{end}}'   # api db web
podman rm -f -t 0 web api db && podman network rm stack
```

Vraag 13 — Reverse proxy tegen CORS

Antwoord. De browser past CORS toe wanneer een pagina een **andere origin** aanroept (schema + host + poort). Komt de pagina van `localhost:18080` en de API van `localhost:8080`, dan verschillen de poorten: andere origin, CORS-headers verplicht. Met een nginx die `/` serveert en `/api/` doorstuurt naar `api:8080`, praat de browser alleen met `localhost:18080`: zelfde origin,

geen CORS. Andere voordelen: de API wordt niet meer gepubliceerd (kleiner aanvalsoppervlak), één plaats voor TLS, compressie, cache en logs, en dezelfde URL `/api/` in ontwikkeling en productie. De regel die het werk doet: `proxy_pass http://api:8080/;` in een blok `location /api/`.

Waarom. CORS is een beleid van de **browser**, niet van de server: een proxy aan de serverkant, die geen browser is, valt er niet onder.

Nuance. De afsluitende `/` in `proxy_pass http://api:8080/` haalt het voorvoegsel `/api/` weg voor het doorsturen: `/api/actuator/health` wordt `/actuator/health`. Zonder die `/` zou de API `/api/actuator/health` ontvangen en `404` antwoorden.

Voorbeeld.

```
curl -s localhost:18080/api/actuator/health     # {"status":"UP"} — via nginx, zonder CORS-header
podman logs web 2>&1 | grep 'GET /api/' | tail -1
```